From Universal Autoencoders to Soft Mixtures of Experts and Style-Conditioned GANs: Image Restoration and Face-to-Sketch Generation

Generative AI – Assignment 1

Shehbaz (21i2525), Section A
Code: https://github.com/Quttoshi/21i2525-A1

Abstract—We build, tune and deploy four related generative systems. Three of them restore 128×128 Oxford-IIIT Pet images corrupted at runtime by salt-and-pepper noise, Gaussian blur or rectangular occlusion: (i) a single universal denoising autoencoder with a genuine compressed bottleneck; (ii) a hard-routing system in which a four-way corruption classifier dispatches each image to one of three specialist autoencoders or to an identity bypass; and (iii) a differentiable soft mixture-of-experts (MoE), initialised from (ii) and fine-tuned jointly with reconstruction, structural-similarity, classification and routing-balance losses. The fourth system is a style-conditioned pix2pix generator that turns FS2K facial photographs into sketches in one of three artistic styles, with a learned style embedding in both the U-Net generator and the PatchGAN discriminator. Every model is tuned with Optuna, tracked in MLflow, exported to ONNX with verified numerical parity, and served by a FastAPI/ONNX-Runtime backend behind a React and Tailwind interface; the application starts with a single Docker Compose command. On 36,690 deterministic test cases the universal autoencoder raises mean PSNR on corrupted inputs from 19.3 dB to 24.6 dB (SSIM 0.63→0.79), and the corruption classifier reaches 99.8% accuracy (macro-F1 0.997). Hard routing and the soft MoE match the universal model on corrupted inputs (24.3 and 24.2 dB) while leaving clean images almost untouched (SSIM 0.997–0.998 vs. 0.847). The learned gate converges to near-hard routing and hedges only on low-severity and mixed corruptions. Because of the deadline, all Optuna budgets are small, and we report them as such.

Index Terms—denoising autoencoder, mixture of experts, image restoration, conditional GAN, pix2pix, Optuna, ONNX

I. Introduction

Real-world images are often degraded in ways the downstream system cannot know in advance. Two broad strategies exist. A single universal model can learn a shared representation that undoes every degradation [1], [2]. Alternatively, the problem can be decomposed: identify the degradation, then apply a specialist. Hard decomposition is interpretable but fragile when the identification is wrong. Soft decomposition through a mixture of experts [3], [4] keeps the specialists and replaces the discrete decision with a learned, differentiable weighting.

This report compares the three strategies under one controlled protocol (Sec. III). It then addresses a different generative problem: paired, style-conditioned image-to-image translation from faces to sketches with a conditional GAN [5], [6]. For each task we describe the alternatives considered, the Optuna search, quantitative and visual results, and the failure cases. Section VIII describes the deployed software product.

II. Related Work

Denoising autoencoders. Vincent et al. [7] showed that reconstructing clean inputs from corrupted ones yields robust representations. Convolutional encoder–decoders with symmetric skip connections (RED-Net [8], U-Net [9]) and residual CNNs (DnCNN [10]) dominate restoration. Their skips, however, carry corrupted detail straight to the output and remove the information bottleneck. The brief forbids unrestricted skips, so we treat skips as an ablation (Sec. IV). Zhao et al. [11] showed that L2 loss correlates poorly with perceived quality and that an L1+MS-SSIM mixture works better, which motivates our α-weighted L1+SSIM objective [12].

All-in-one restoration. AirNet [1] learns a degradation representation by contrastive learning, and PromptIR [2] conditions a restoration network on learned prompts. Both show that knowing the degradation helps, which motivates the explicit classifier of Task 2 and the gate of Task 3.

Mixtures of experts. Jacobs et al. [3] introduced gated mixtures of local experts. Sparse gating [4] and Switch Transformers [13] added load-balancing auxiliary losses because gates tend to collapse onto a few experts. Our balance term follows the same reasoning. A temperature-scaled softmax [14] controls how sharp the routing is.

Conditional GANs. pix2pix [6] pairs a U-Net generator with a PatchGAN discriminator and an L1 term ($\lambda = 100$). Class conditioning can be injected by concatenation, feature-wise modulation (FiLM [15]) or projection [16]. FS2K [17] is the largest public facial-sketch dataset with three artist styles.

Tooling. Optuna [18] provides define-by-run search with TPE [19], median pruning and fANOVA importance [20]. MLflow [21] records experiments, and ONNX [22] provides framework-independent inference.

III. Data and Corruption Protocol

Split. The official Oxford-IIIT Pet [23] trainval list (3,680 images) is permuted with seed 42 and split 80/20

TABLE I
Corruption configuration (training sampled at runtime; test fixed).

Condition	Training (per load)	Test levels L / M / H
Clean	image unchanged	–
Salt & pepper	$p \sim \mathcal{U}(0.02, 0.15)$; each hit pixel black or white w.p. $\frac{1}{2}$	$p = 0.03/0.08/0.15$
Gaussian blur	$k \in \{3, 5, 7\}$, $\sigma \sim \mathcal{U}(0.5, 2.5)$, reflect pad	$(3, 0.7)/(5, 1.5)/(7, 2.5)$
Occlusion	1–3 black non-overlapping rectangles, total area $\sim \mathcal{U}(10\%, 35\%)$, aspect $\in [\frac{1}{2}, 2]$, random position	1 / 2 / 3 rects, 10 / 20 / 35%

into 2,944 training and 736 validation images. The 3,669 official test images are used only once, for final evaluation. Images are converted to RGB and resized to 128×128 (bicubic). The same split is shared by Tasks 1–3.

Runtime corruption. Corruption is applied inside the Dataset.__getitem__. Each time an image is loaded, a condition is drawn uniformly from {clean, salt-and-pepper, blur, occlusion} together with fresh parameters (Table I); no corrupted copy is ever stored. Our BalancedBatchSampler goes further than equal probability: every batch contains exactly $B/4$ images of each condition, which the classifier (Task 2) and the balance loss (Task 3) both rely on. Because DataLoader workers are spawned processes, the NumPy generator is seeded from torch's per-worker RNG, which avoids the well-known duplicate-augmentation bug.

Occlusion sampling. The brief requires the masks to jointly cover 10–35%. With overlapping rectangles the union area is hard to control, so we place rectangles without overlap. The union then equals the sum of the parts, and coverage matches the target to within rounding (unit-tested to $\pm 1\%$).

Deterministic manifests. Validation and test corruptions are generated once and stored as JSON. Each entry records the image, condition, severity, per-entry seed, salt-and-pepper pixel seed, blur (k, σ) and rectangle coordinates. The validation manifest assigns one condition per image, balanced (184 per condition). The test manifest holds clean plus three levels of each corruption for every test image, i.e. $3{,}669 \times 10 = 36{,}690$ cases. Replaying an entry reproduces the corrupted image bit-exactly. Our blur matches torchvision to $< 10^{-5}$.

IV. Task 1: Universal Denoising Autoencoder

A. Architecture

The encoder E (Fig. 1) is a stem convolution followed by n stride-2 stages (conv-BN-LeakyReLU $\times 2$) with channels $c, 2c, 4c, 8c$. A 1×1 convolution with tanh then projects the features to a latent $z \in \mathbb{R}^{C_z \times r \times r}$. The decoder D mirrors the encoder with bilinear upsampling plus convolution, which avoids the checkerboard artefacts of transposed convolutions [24], and ends in a sigmoid. We chose a spatial latent over a fully connected vector because it keeps object layout while still forcing a large compression. With $r \in \{8, 16\}$ and $C_z \in \{4, \ldots, 32\}$ the latent holds 256–8,192 values, against 49,152 input values ($6\times$–$192\times$ compression). A preliminary run showed the 8×8 latent saturating near 20 dB, so the latent resolution became an Optuna variable rather than a fixed choice.

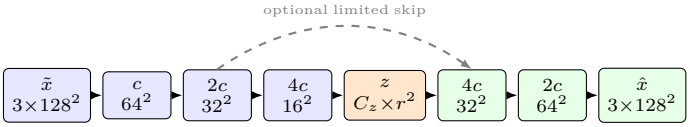


Fig. 1. Universal DAE ($r = 16$ variant shown; $r = 8$ adds one $8c$ stage). Stride-2 conv stages compress the input to a 1×1-projected spatial latent; bilinear-upsampling stages decode it. The dashed skip is used only in the ablation.

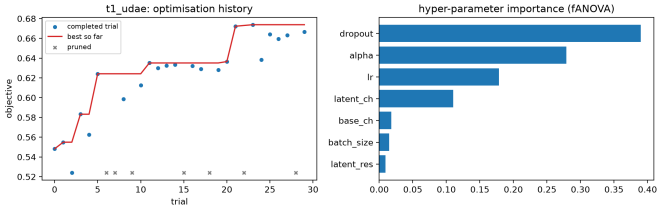


Fig. 2. Task 1 Optuna study: optimisation history (left) and fANOVA importance (right).

B. Loss and training

The model minimises $\mathcal{L}_{\text{UDAE}} = \alpha\,\mathcal{L}_{L1}(x, \hat{x}) + (1 - \alpha)(1 - \text{SSIM}(x, \hat{x}))$. We train with AdamW [25], a one-cycle schedule [26], mixed precision (the loss is computed in FP32 because SSIM is numerically fragile in FP16), horizontal-flip augmentation and early stopping on the validation objective $J = \frac{1}{2}\,\text{SSIM} + \frac{1}{2}\,\text{PSNR}/40$. Dividing PSNR by 40 dB brings it to a range comparable with SSIM, so neither term dominates the objective.

C. Hyper-parameter optimisation

Optuna's TPE sampler (seed 42) with a median pruner searched learning rate, batch size, latent resolution, latent channels (bottleneck dimension), encoder base channels, dropout and α (Table II), with shortened 10-epoch trials. Of 30 trials, 23 completed and 7 were pruned; the best (trial 23) reached $J = 0.674$, up from 0.55 for the first trials (Fig. 2). fANOVA ranks learning rate, dropout and α as most important; latent resolution mattered once the 8×8 option was shown to plateau, and every top trial used $r = 16$. The selected $\alpha = 0.64$ gives SSIM noticeably more weight than the brief's starting point of 0.8, consistent with [11]: the structural term sharpens edges that pure L1 blurs.

D. Results

The selected configuration was retrained for 60 epochs (Fig. 3); validation PSNR rose from 19 dB to 25.3 dB with

TABLE II
Task 1 Optuna search space and selected configuration.

Parameter	Range	Selected
learning rate	$[10^{-4}, 3{\cdot}10^{-3}]$ log	$1.78{\cdot}10^{-3}$
batch size	{16, 32, 64}	16
latent resolution r	{8, 16}	16
latent channels C_z	{4, 8, 16, 32}	32 (8,192 values)
base channels c	{16, 32, 48}	32
dropout	$[0, 0.3]$	0.079
α (L1 weight)	$[0.5, 0.95]$	0.636

TABLE III
Task 1 test results (3,669 images × 10 conditions). "In" = corrupted input vs. clean target.

Condition	Sev.	PSNR in	PSNR out	SSIM in	SSIM out
Clean	–	∞	26.48	1.000	0.847
S&P	L	20.13	26.47	0.603	0.847
	M	15.87	26.37	0.339	0.842
	H	13.14	26.10	0.204	0.831
Blur	L	32.34	26.63	0.944	0.844
	M	26.77	26.19	0.806	0.828
	H	24.35	24.59	0.692	0.736
Occlusion	L	16.74	23.63	0.862	0.796
	M	13.30	21.80	0.727	0.744
	H	10.77	19.82	0.537	0.665
All corrupted		19.27	24.62	0.635	0.792

no sign of over-fitting, because the runtime corruption pipeline never shows the same corrupted image twice. Table III reports the untouched test set.

Analysis. Salt-and-pepper is restored almost perfectly regardless of severity: output PSNR stays at about 26.3 dB while input PSNR falls from 20.1 to 13.1 dB, so the gain grows to +13 dB at $p = 0.15$. Impulse noise is spatially independent and is precisely what a bottleneck discards. Occlusion is hardest (21.7 dB). The model has to invent missing content, and with no adversarial or perceptual loss it fills the masks with smooth, locally averaged colour. The most instructive result is the ceiling: clean and lightly blurred inputs come out at about 26.5 dB, i.e. the autoencoder degrades images that need no restoration (light blur: 32.3→26.6 dB). A bottleneck of 8,192 values cannot carry all high-frequency detail of a 49,152-value image. This is the trade-off the brief asks for: a meaningful bottleneck removes noise but also removes texture. It directly motivates the identity bypass of Task 2 and the identity branch of Task 3.

Fig. 4 shows twelve representative cases (one per condition and severity, plus extras) and Fig. 5 six failure cases, each with clean target, input, output and absolute-error map. The failures fall into four groups. (1) Hallucinated texture: on fine grass and flowers under heavy S&P or blur, the decoder replaces true high-frequency texture with a mottled pattern, and the error map lights up uniformly across the background. (2) Occluder on a dark background: the network has learned that black rectangles are corruption, so it brightens them, but here

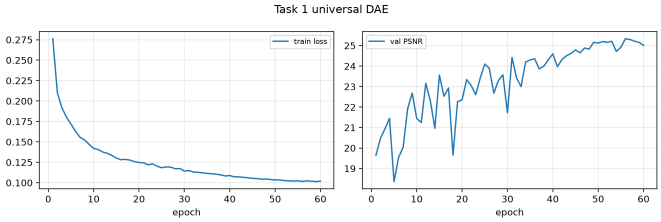


Fig. 3. Task 1 final training: training loss and validation PSNR per epoch (MLflow).

the true content is black and the output gains a grey halo (bright border in the error map). (3) Large multi-rectangle occlusion: three masks covering 35% leave little context, and the filled regions become flat, blurry blobs. (4) Heavy blur on detailed scenes: $(7, 2.5)$ removes information that cannot be recovered, and the output stays soft (SSIM 0.74).

Skip connections. The final model uses no skips. The architecture supports limited additive skips from the deepest encoder stages (Fig. 1, dashed), but the planned ablation was not run before the deadline. Our expectation, from the error analysis above, is that a 32×32 skip would raise the clean and light-blur ceiling but would also pass occlusion masks and impulse noise straight through. That is why skips are excluded from the bottleneck requirement.

V. Task 2: Classifier and Hard-Routed Specialists

A. Corruption classifier

The classifier stacks four stages of [conv, stride-2 conv] (Fig. 7). It applies no pooling before stride, because salt-and-pepper noise and blur are high-frequency signatures that early pooling would average away. The head concatenates global average and global max pooling: max pooling responds to sparse saturated pixels (S&P) and to large flat black regions (occlusion). Training uses cross-entropy on the balanced runtime batches. Optuna tuned learning rate, batch size, channel configuration, dropout and weight decay, maximising validation macro-F1.

B. Specialists

Three autoencoders with the Task 1 architecture are trained independently, each only on its own corruption with the clean image as target. To keep the cost feasible, a single shared Optuna study selects the common architecture and optimiser settings (learning rate, latent size, channels, batch size, α) by maximising the mean validation objective of the three specialists. The three final models are then trained from scratch with independent parameters.

Short-horizon bias (a difficulty we hit). The first shared study used 6-epoch trials (15 trials, 10 completed). It selected a tiny 4-channel latent (1,024 values), which converges quickly but plateaus: after full training the specialists reached only 24.3/24.3/20.7 dB

TABLE IV
Task 2 classifier on the test manifest.

Class	Precision	Recall	F1	Support
Clean	0.998	0.984	0.991	3,669
Salt & pepper	1.000	1.000	1.000	11,007
Blur	0.997	1.000	0.998	11,007
Occlusion	0.998	1.000	0.999	11,007
Macro avg	0.998	0.996	0.997	36,690

on S&P/blur/occlusion validation, about 2 dB below the universal model, even though each specialist faces an easier problem. This is the known short-horizon bias of multi-fidelity tuning: short trials reward fast early progress rather than final quality. We therefore ran a second study with 15-epoch trials, warm-started with the Task 1 configuration. Its best trial (the warm start, 32-channel latent) scored 0.689, against 0.585 for the short study (Fig. 6). All three specialists were then retrained with this configuration (20 epochs each, compute-limited). Validation PSNR/SSIM improved from 24.3/0.74, 24.3/0.73 and 20.7/0.65 to 26.1/0.82, 25.9/0.81 and 21.3/0.71 for S&P, blur and occlusion. The evidence from the second study therefore fixed a 1.5–2 dB deficit that the first study had introduced.

C. Hard routing

At inference the class $r = \arg\max_k p_k$ selects $\hat{x} = \tilde{x}$ (identity bypass for clean) or $A_r(\tilde{x})$. We evaluate oracle routing (label from the test manifest), which measures specialist capability, and predicted routing, which measures the deployed system.

D. Results

Classifier. Its Optuna study (25 trials, 12 completed, 13 pruned) selected learning rate $2.7{\cdot}10^{-4}$, batch 32, channels 32-64-128-256, dropout 0.07 and weight decay $1.7{\cdot}10^{-4}$ (validation macro-F1 0.989 in 6 epochs). After 40 epochs it reached 99.7% validation accuracy. On all 36,690 test cases it achieves 99.82% accuracy, macro precision 0.998, macro recall 0.996 and macro-F1 0.997 (Table IV, Fig. 8). S&P is perfect: isolated saturated pixels are unmistakable to the max-pooling branch. Nearly all errors are clean images predicted as occlusion (0.71%) or blur (0.84%); low-severity blur and occlusion are the only corrupted classes below 100%.

Routing. Because the classifier is so accurate, predicted and oracle routing differ by at most 0.02 dB on every corrupted cell; overall corrupted PSNR is 24.31 (predicted) vs. 24.31 dB (oracle), SSIM 0.778 (Table V). Clean inputs pass through the identity bypass unchanged in 98.4% of cases (clean SSIM 0.997 vs. 0.847 for Task 1), which removes Task 1's main weakness. On corrupted inputs Task 2 is slightly below Task 1 (24.3 vs. 24.6 dB). Since the oracle row is identical, this is not a routing problem. The specialists simply received 20 training epochs against Task 1's 60, which shows that a hard-routed system is only as good as its experts.

Classifier-induced failures (Fig. 9). The worst routing errors are all clean images with large uniform black regions: black studio backgrounds, letterbox bars, a dark car interior. The classifier sees large zero-valued areas and predicts occlusion. The occlusion expert, trained only to fill black rectangles, then "inpaints" the legitimate black background with grey or white and blurs the subject, so PSNR collapses from ∞ (identity) to 12–16 dB. Saturated, colourful clean images (a purple backdrop, rainbow balloons) are occasionally routed to blur and suffer colour shifts. These cases show the fragility of a hard decision: a 1.6% error rate on clean images produces the largest per-image losses in the whole task. A soft gate that hedges between the identity and an expert, or an explicit "uncertain" fallback to identity, would bound the damage.

VI. Task 3: Soft Mixture-of-Experts

A. Model

The gate G is the Task 2 classifier, and its logits give $w = \mathrm{softmax}(G(\tilde{x})/\tau)$ over [identity, S&P, blur, occlusion]. The output is $\hat{x} = w_0\tilde{x} + \sum_{k=1}^{3} w_k A_k(\tilde{x})$. Branch 0 matches the classifier's clean class, so the initial gate is already meaningful. The whole pipeline, including the identity branch and the weighted sum, is exported as one ONNX graph with outputs (restored, weights, logits).

B. Training

Joint training does not start from random components. The gate and experts are loaded from Task 2. (1) Warm-up: the experts are frozen (parameters and BatchNorm statistics) and only the gate trains for two epochs, adapting to the temperature and to the reconstruction signal. (2) Joint fine-tuning: all parameters train at a smaller learning rate. The loss is $\mathcal{L}_{\mathrm{MoE}} = \lambda_1\mathcal{L}_{L1} + \lambda_s(1-\mathrm{SSIM}) + \lambda_c\mathcal{L}_{CE} + \lambda_b\mathcal{L}_{\mathrm{balance}}$, where $\mathcal{L}_{CE} = -\log w_y$ is computed on the temperature-scaled distribution that is actually used for mixing, and $\mathcal{L}_{\mathrm{balance}} = \sum_k(\bar{w}_k - \frac{1}{4})^2$. Because every batch is exactly class-balanced, perfect routing gives $\bar{w}_k = \frac{1}{4}$. The balance term therefore penalises collapse without penalising correct specialisation. A per-sample entropy bonus, by contrast, would discourage confident correct routing, which is why we rejected it. This is the same motivation as the load-balancing losses of [4], [13].

C. Optuna

The study searches the joint learning rate, τ, λ_c, λ_b and λ_1 (with $\lambda_s = 1 - \lambda_1$). A custom pruning rule stops any trial in which some branch's mean validation weight falls below 0.02 (routing collapse), in addition to median pruning.

TABLE V
Restoration methods on the test set (PSNR dB / SSIM).

Condition	T1 universal	T2 hard (pred.)	T3 soft MoE
Clean	26.48 / 0.847	$\approx\infty$ / 0.997	$\approx\infty$ / 0.998
Salt & pepper	26.31 / 0.840	25.89 / 0.824	25.61 / 0.819
Blur	25.80 / 0.803	25.44 / 0.796	25.40 / 0.792
Occlusion	21.75 / 0.735	21.59 / 0.713	21.47 / 0.703
All corrupted	24.62 / 0.792	24.31 / 0.778	24.16 / 0.771

D. Optuna and training

Four trials (compute-limited; 1 warm-up epoch and 2 joint epochs each) all completed without routing collapse. The clearest effect was temperature: the two sharp gates ($\tau = 0.34$ and 0.60) scored 0.905 and 0.900, against 0.883 and 0.847 for $\tau \approx 2.7$. With a 99.8%-accurate classifier, decisive routing pays off. Selected: $\tau = 0.34$, $\lambda_c = 0.43$, $\lambda_b = 0.006$, $\lambda_1 = 0.82$. Before any fine-tuning, the classifier-initialised gate already routed 99.7% of validation images correctly with balanced mean weights (0.23–0.26), which confirms that initialising from Task 2 avoids a random-gate collapse.

E. Results

Table V compares the three restoration systems on the test manifest.

Gate behaviour. The mean-weight heatmap (Fig. 10) is almost perfectly diagonal: at medium and high severity the correct branch receives $\geq$0.999 of the weight, and 99.0% of all test images have one branch with $w \geq 0.9$. The top-weighted branch matches the true corruption for 99.6% of images. No expert is inactive: mean weights are 0.30/0.30/0.30 for the experts and 0.10 for the identity, which matches the 3:3:3:1 class mix of the test manifest. No expert dominates unrelated inputs: the mean weight a branch receives on inputs it is not responsible for is at most 0.005. The only real hedging happens at low severity (Fig. 12): faint blur, sparse noise or a small occluder produce near 50/50 splits between the identity and the matching expert. That is the sensible output when the corruption barely changes the image.

Mixed corruptions. On 300 test images with two simultaneous medium corruptions, which none of the models saw in training, the universal DAE is best on every pair (blur+occlusion 21.9, S&P+blur 22.3, S&P+occlusion 21.6 dB). For S&P+occlusion, the gate splits 0.84/0.16 between the S&P and occlusion experts, and this blend beats hard routing (14.5 vs. 13.6 dB). It still falls far short of the universal model, because each expert has only seen one corruption and treats the other as signal.

Interpretation. With a near-perfect classifier and a strong CE term, joint fine-tuning keeps the gate close to its initialisation. The soft MoE then behaves as a smoothed hard router: it matches hard routing in distribution, hedges where it should, and is fully differentiable, but it cannot exceed the quality of its experts. A longer joint stage with a smaller λ_c, or experts trained on mixtures, would be needed to exploit the soft combination.

VII. Task 4: Style-Conditioned Face-to-Sketch cGAN

A. Data

We use the official FS2K [17] split (1,058 train / 1,046 test). Fifteen percent of the training portion is held out with seed 42, stratified by style (899/159; per-style 303/298/298 and 54/52/53). Photos and sketches are resized to 128×128. The sketch path is derived from the photo path exactly as in the official split script, which guarantees correct pairing. Augmentation upsamples both images to 142^2, takes the same random crop and applies the same flip. Independent transforms would destroy the pixel correspondence that the L1 term relies on.

B. Architecture

Generator $G(x, s)$: a 7-level U-Net [9] ($128 \to 1$) with full skip connections, as in pix2pix. Skips are appropriate here, unlike in Task 1, because the output must stay spatially aligned with the input. A learned embedding $e_s = \mathrm{Emb}(s) \in \mathbb{R}^d$ enters the network in two ways: it is concatenated to the 1×1 bottleneck, and it is mapped to per-channel FiLM [15] scale and shift in every decoder block (zero-initialised, so training starts from the unconditional model). Bottleneck concatenation lets the style affect global layout, and FiLM lets it change stroke texture at every scale. We prefer this to input-only concatenation, which the encoder can learn to ignore. Dropout in the first three decoder blocks provides stochasticity. Discriminator $D(x, y, s)$: a 70×70 PatchGAN on (x, y) concatenated with a projected, spatially tiled style map, so a sketch counts as "real" only for its own style.

C. Objective and training

$\mathcal{L}_D = \frac{1}{2}[\mathrm{BCE}(D(x, y, s), 1) + \mathrm{BCE}(D(x, G(x, s), s), 0)]$ and $\mathcal{L}_G = \mathrm{BCE}(D(x, G(x, s), s), 1) + \lambda_{L1}\|y - G(x, s)\|_1$, optimised with Adam ($\beta_1 = 0.5$), a constant learning rate for the first half of training and linear decay to zero over the second half, and pix2pix initialisation. We log D_{real}, D_{fake}, G_{adv}, G_{L1} and validation L1/PSNR/SSIM separately. A fixed set of eight validation photographs is rendered every five epochs in its own style and in all three styles.

D. Optuna

The search space covers generator and discriminator learning rates, batch size, base channels, dropout, embedding dimension and $\lambda_{L1} \in [10, 200]$ (log). Four 6-epoch trials (compute-limited) were run, using the validation objective $\frac{1}{2}\mathrm{SSIM} + \frac{1}{2}(1 - \mathrm{L1})$. The best (0.666) used $\mathrm{lr}_G = 2{\cdot}10^{-4}$, a discriminator about 3× slower ($\mathrm{lr}_D = 6{\cdot}10^{-5}$), batch 4, 48 base channels, dropout 0.15, a 16-dimensional style embedding and $\lambda_{L1} = 44$. The two trials in which the discriminator learned faster than the generator scored

TABLE VI
Task 4 on the official FS2K test split.

Style	L1 ↓	PSNR ↑	SSIM ↑	FID ↓	n
Style 1	0.084	16.98	0.493	91.9	619
Style 2	0.155	12.35	0.370	76.9	381
Style 3	0.070	18.19	0.592	–	46
Overall	0.109	15.35	0.453	78.2	1,046

lowest (0.63), consistent with the usual advice not to let D overpower G. The selected configuration was retrained for 40 epochs (reduced from the planned 150 because of the deadline).

E. Results

Training dynamics (Fig. 13). Training was stable: D_{real} and D_{fake} stayed balanced around 0.55–0.58 for all 40 epochs, with no discriminator domination and no mode collapse. G_{L1} fell from 0.30 to 0.19 and validation SSIM rose from 0.41 to 0.47. The fixed validation grid (Fig. 14) shows the generator's progress: by epoch 40 it produces clean sketches that keep identity, hairstyle and accessories such as caps and earrings.

Style conditioning. The style sweep in Fig. 14 shows that the embedding controls the output rather than being ignored: Style 1 gives light, thin outlines; Style 2 gives dark, heavy hair and shading strokes; and Style 3 gives a grey, paper-like background with soft shading. Each matches its artist's style in the training data.

Metrics (Table VI). Pixel metrics and FID disagree in an informative way. Style 2, with heavy strokes, has the worst L1/SSIM but the best FID: dense dark strokes are almost impossible to reproduce at the same pixel positions, yet their overall statistics are easy to imitate. Style 3's sketches are mostly flat grey and score best on pixel metrics. FID is not reported for Style 3 because only 46 test images exist (the official test split is heavily unbalanced, 619/381/46), and a 2048-dimensional covariance estimate from 46 samples would be meaningless.

Failure cases (Fig. 15). The lowest-SSIM test images are all subjects with long, voluminous hair. The artist draws hair as dense individual strokes, and the generator produces plausible strands that are not at the same pixel positions. The sketches look reasonable, but SSIM drops to 0.16–0.18. This is the main weakness of paired pixel losses for line art. A second failure type is out-of-distribution faces: heavy face paint (the last row of Fig. 14) breaks the facial structure and produces fragmented strokes. Very light blonde hair against a light background is sometimes drawn too faintly.

VIII. Application and Deployment

Design. The interface was first designed in Google Stitch (Fig. 17): a top navigation bar with the four named workspaces, a 340px control column and a results area. The design tokens (indigo accent, 16px cards, Inter typeface) were carried over to Tailwind's @theme.

Backend. FastAPI loads the seven ONNX models at start-up with ONNX Runtime and warms them up. It exposes /api/health, /api/info, /api/samples, /api/corrupt, /api/restore/{universal,hard,soft} and /api/sketch. Uploads are validated by MIME type, size (≤10 MB), decodability and minimum size, then EXIF-transposed and resized. The corruption code used for runtime corruption in the app is the same module used for training (NumPy-only, so the backend image needs no PyTorch). Responses carry the images, the corruption parameters, classifier probabilities or routing weights, PSNR when a clean reference is known, and timing broken down by stage.

Frontend. React 19, TypeScript and Tailwind 4 (Vite). Each workspace supports sample selection, drag-and-drop upload (and webcam capture for faces), corruption type, severity or custom parameters with a seed, oracle versus predicted routing, probability and weight bars, a stacked contribution bar, error maps and result download.

Containers. docker compose up –build starts the backend (Python 3.12 slim, uv-locked dependencies, health check) and the frontend (multi-stage Node build served by nginx, which proxies /api). Models are mounted read-only from ./models. The backend reports every expected model file in /api/health; a workspace whose model is missing fails gracefully with HTTP 503 and is listed as "missing" on the Overview page. Eleven API tests (validation, path-traversal protection, oracle versus predicted routing, weights summing to one, all three sketch styles) run against ONNX models with the production signatures.

ONNX parity. Each exported model was run on real validation images in PyTorch and ONNX Runtime (CPU). Maximum absolute differences: universal DAE 3.6×10^{-6}; classifier logits 3.3×10^{-6}; specialists 2.4×10^{-6} (S&P), 1.5×10^{-6} (blur) and 1.2×10^{-5} (occlusion); soft MoE 3.7×10^{-6} (restored), 5×10^{-10} (weights); sketch generator 2.0×10^{-6}. All are within float32 round-off.

IX. Limitations

(1) Short schedules: the specialists (20 epochs), the MoE joint stage and the GAN were trained with deadline-reduced budgets, and the Task 3 and 4 Optuna studies used only four trials. (2) Budgets: Optuna trials were short (6–15 epochs) on a 4 GB laptop GPU, and we observed short-horizon bias directly. (3) Bottleneck ceiling: about 26.5 dB at 128×128; the planned skip ablation was not run. (4) Occlusion is inpainting, and L1+SSIM produces blurry fills. A perceptual or adversarial term would help. (5) Engineering: on Windows, every DataLoader worker re-imports CUDA (about 2 GB commit each). Leaked persistent workers stalled training until we added explicit worker shutdown. Per-sample GPU→CPU transfers made evaluation take over 2 h until we batched them (90 s after the fix).

X. Conclusion

A single autoencoder with a genuine bottleneck removes impulse noise very effectively (+13 dB at $p = 0.15$), handles occlusion moderately and caps clean-image fidelity at about 26.5 dB. An explicit corruption classifier is almost perfect (99.8%), so hard routing behaves like oracle routing, and the identity bypass removes the clean-image penalty. But hard routing is only as good as its experts, and its rare errors (black backgrounds mistaken for occlusion) are its most damaging failures. The jointly trained soft MoE learns near-hard routing, hedges only on low-severity and mixed inputs, and has no inactive or dominating experts. Its potential advantage depends on experts that see corruption mixtures. The style-conditioned pix2pix trains stably and produces three visibly distinct sketch styles from one generator (overall FID 78.2). Its failures, misaligned hair strokes, expose the limits of paired pixel losses for line art. All code, configurations, Optuna studies, MLflow records and ONNX models are in the repository.

References

[1] B. Li, X. Liu, P. Hu, Z. Wu, J. Lv, and X. Peng, "All-in-one image restoration for unknown corruption," in Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR), 2022, pp. 17 452–17 462.

[2] V. Potlapalli, S. W. Zamir, S. Khan, and F. S. Khan, "PromptIR: Prompting for all-in-one blind image restoration," in Advances in Neural Information Processing Systems (NeurIPS), 2023.

[3] R. A. Jacobs, M. I. Jordan, S. J. Nowlan, and G. E. Hinton, "Adaptive mixtures of local experts," Neural Computation, vol. 3, no. 1, pp. 79–87, 1991.

[4] N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, G. Hinton, and J. Dean, "Outrageously large neural networks: The sparsely-gated mixture-of-experts layer," in Int. Conf. Learning Representations (ICLR), 2017.

[5] M. Mirza and S. Osindero, "Conditional generative adversarial nets," arXiv preprint arXiv:1411.1784, 2014.

[6] P. Isola, J.-Y. Zhu, T. Zhou, and A. A. Efros, "Image-to-image translation with conditional adversarial networks," in Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR), 2017, pp. 1125–1134.

[7] P. Vincent, H. Larochelle, Y. Bengio, and P.-A. Manzagol, "Extracting and composing robust features with denoising autoencoders," in Proc. Int. Conf. Machine Learning (ICML), 2008, pp. 1096–1103.

[8] X.-J. Mao, C. Shen, and Y.-B. Yang, "Image restoration using very deep convolutional encoder-decoder networks with symmetric skip connections," in Advances in Neural Information Processing Systems (NeurIPS), 2016.

[9] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional networks for biomedical image segmentation," in Medical Image Computing and Computer-Assisted Intervention (MICCAI), 2015, pp. 234–241.

[10] K. Zhang, W. Zuo, Y. Chen, D. Meng, and L. Zhang, "Beyond a Gaussian denoiser: Residual learning of deep CNN for image denoising," IEEE Trans. Image Processing, vol. 26, no. 7, pp. 3142–3155, 2017.

[11] H. Zhao, O. Gallo, I. Frosio, and J. Kautz, "Loss functions for image restoration with neural networks," IEEE Trans. Computational Imaging, vol. 3, no. 1, pp. 47–57, 2017.

[12] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, "Image quality assessment: From error visibility to structural similarity," IEEE Trans. Image Processing, vol. 13, no. 4, pp. 600–612, 2004.

[13] W. Fedus, B. Zoph, and N. Shazeer, "Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity," Journal of Machine Learning Research, vol. 23, no. 120, pp. 1–39, 2022.

[14] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," in NIPS Deep Learning Workshop, 2015.

[15] E. Perez, F. Strub, H. de Vries, V. Dumoulin, and A. Courville, "FiLM: Visual reasoning with a general conditioning layer," in Proc. AAAI Conf. Artificial Intelligence, 2018.

[16] T. Miyato and M. Koyama, "cGANs with projection discriminator," in Int. Conf. Learning Representations (ICLR), 2018.

[17] D.-P. Fan, Z. Huang, P. Zheng, H. Liu, X. Qin, and L. Van Gool, "Facial-sketch synthesis: A new challenge," Machine Intelligence Research, vol. 19, no. 4, pp. 257–287, 2022.

[18] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, "Optuna: A next-generation hyperparameter optimization framework," in Proc. ACM SIGKDD Int. Conf. Knowledge Discovery & Data Mining, 2019, pp. 2623–2631.

[19] J. Bergstra, R. Bardenet, Y. Bengio, and B. Kégl, "Algorithms for hyper-parameter optimization," in Advances in Neural Information Processing Systems (NeurIPS), 2011.

[20] F. Hutter, H. Hoos, and K. Leyton-Brown, "An efficient approach for assessing hyperparameter importance," in Proc. Int. Conf. Machine Learning (ICML), 2014, pp. 754–762.

[21] M. Zaharia et al., "MLflow: An open source platform for the machine learning lifecycle," https://mlflow.org, 2018.

[22] ONNX Community, "Open neural network exchange," https://onnx.ai, 2024.

[23] O. M. Parkhi, A. Vedaldi, A. Zisserman, and C. V. Jawahar, "Cats and dogs," in Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR), 2012, pp. 3498–3505.

[24] A. Odena, V. Dumoulin, and C. Olah, "Deconvolution and checkerboard artifacts," Distill, 2016, doi:10.23915/distill.00003.

[25] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in Int. Conf. Learning Representations (ICLR), 2019.

[26] L. N. Smith and N. Topin, "Super-convergence: Very fast training of neural networks using large learning rates," in Artificial Intelligence and Machine Learning for Multi-Domain Operations Applications, vol. 11006. SPIE, 2019.

Appendix A
Use of AI Assistants

Tool. Claude Code (Anthropic, Claude Opus model), an AI coding assistant, used inside VS Code.

Used for. Reading the brief into a requirements checklist; drafting the data pipeline, models, training and Optuna scripts, evaluation, ONNX export, FastAPI backend, React UI, Dockerfiles, README, the Stitch prompts and this report; debugging; looking up the FS2K split layout and the SciPy/Tailwind API changes.

Verification and corrections. (i) Unit tests check every corruption against the specification (S&P fraction and 50/50 colours, occlusion coverage ±1%, blur equal to torchvision to $< 10^{-5}$) and the determinism of the manifests. Eleven API tests cover the backend. (ii) Every eval and export script was dry-run on untrained models before the real runs; this caught four bugs: infinite PSNR on identity-restored clean images inflating averages, an empty-grid crash, a removed SciPy argument, and a network-dependent FID failure. (iii) ONNX parity is checked numerically for every model. (iv) Figures of corruptions, FS2K pairs, failures and misroutes were inspected visually. (v) Experiment results led to design corrections: the 8×8 latent plateau (latent resolution added to the search), short-horizon bias (second specialist

study), Windows worker leaks and per-sample GPU syncs (fixed and documented above). The author reviewed the generated code and is responsible for it.

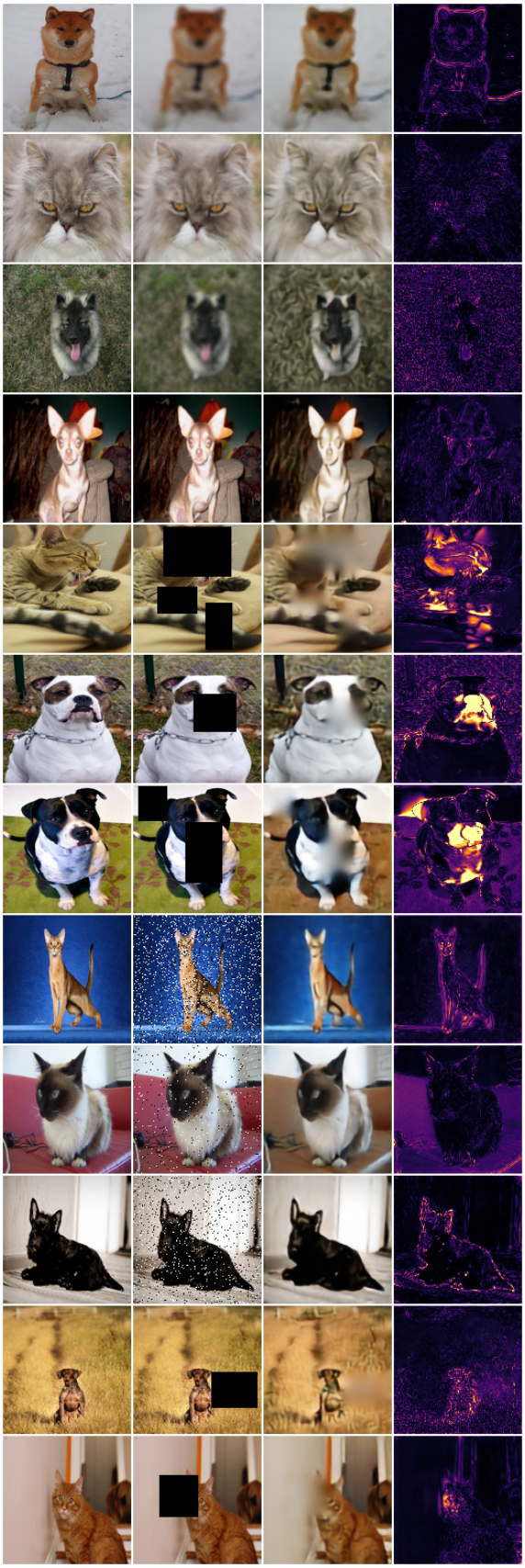


Fig. 4. Task 1 representative test examples. Columns: clean target, corrupted input, restoration, |error| map.

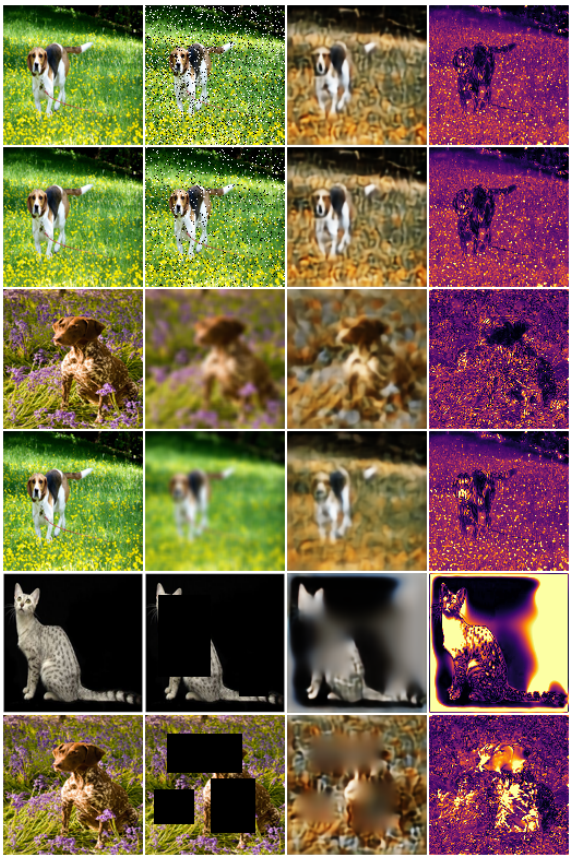


Fig. 5. Task 1 failure cases (lowest SSIM per corruption): hallucinated texture under S&P and blur, a dark-background occlusion and heavy multi-rectangle occlusion.

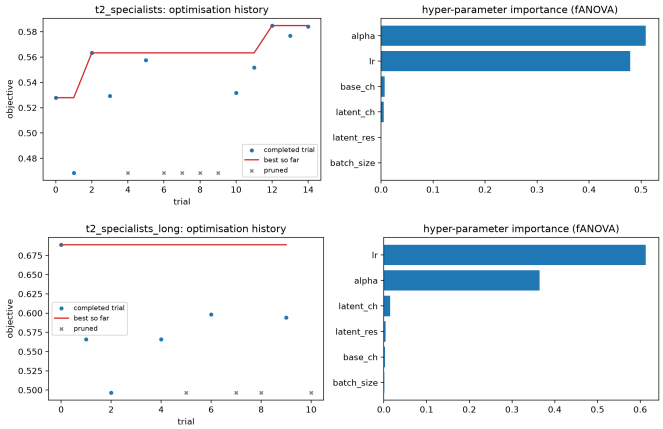


Fig. 6. Shared specialist studies: 6-epoch trials (top) versus 15-epoch trials, warm-started (bottom).

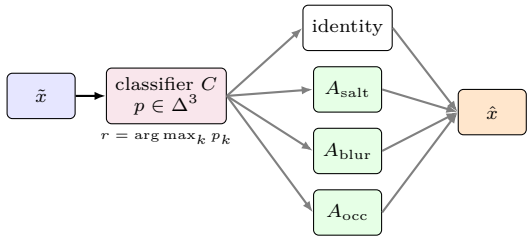


Fig. 7. Hard routing: exactly one branch is executed per image.

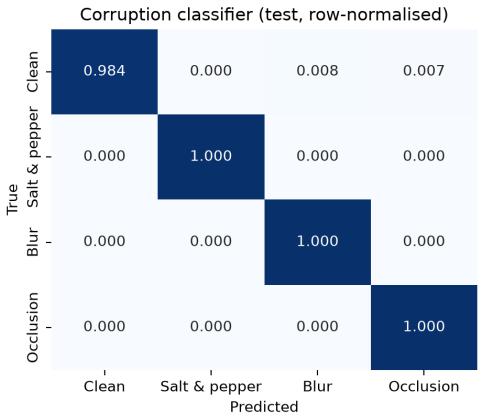


Fig. 8. Row-normalised confusion matrix of the corruption classifier (test).

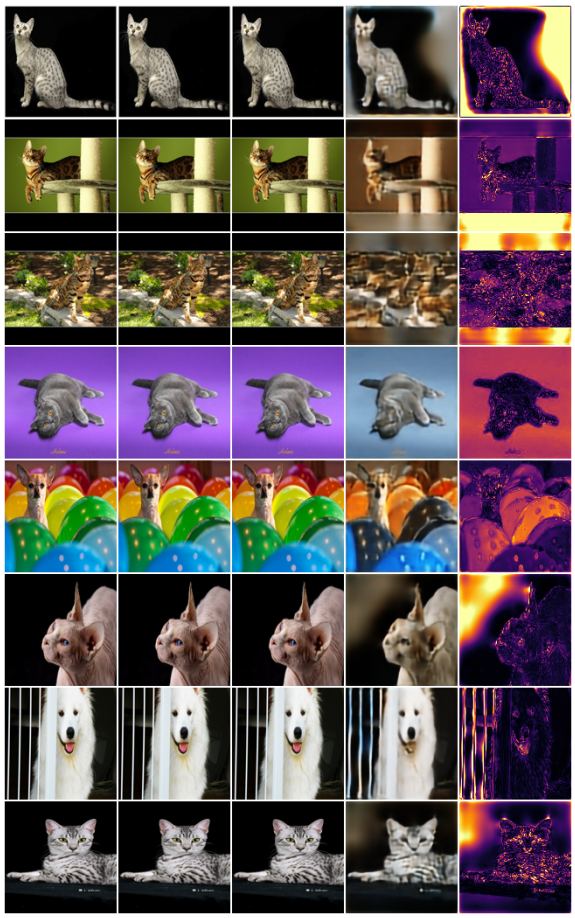


Fig. 9. Misrouting failures. Columns: clean, input (clean), oracle output (identity), predicted-route output, error. Uniform black backgrounds are mistaken for occlusion.

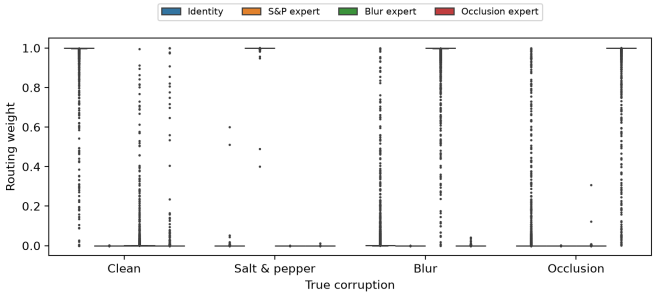


Fig. 11. Distribution of routing weights per true corruption.

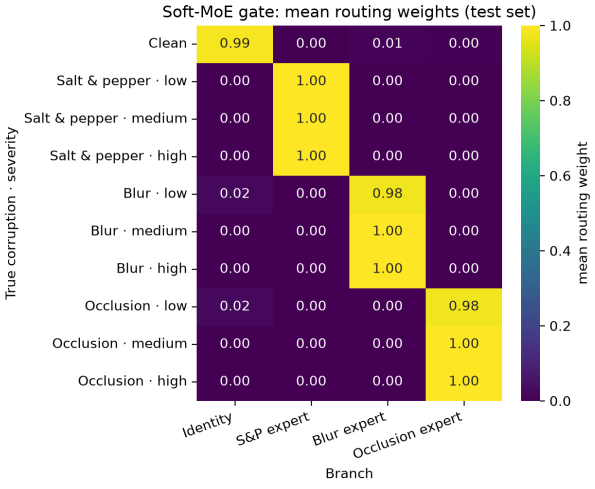


Fig. 10. Mean routing weight per true corruption and severity (test set).



Fig. 12. Dominant-expert examples (top) and the most distributed examples (bottom). Columns: clean, input, identity, S&P, blur and occlusion expert outputs, MoE output, error. The distributed cases are low-severity inputs split about 50/50 between the identity and the matching expert.

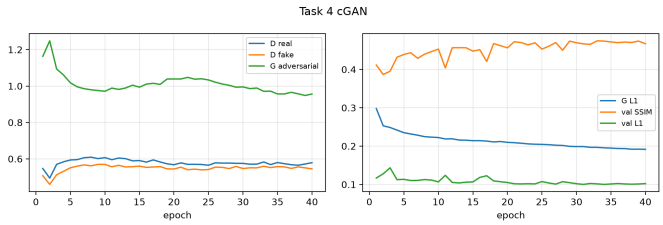


Fig. 13. Task 4 training: discriminator real/fake loss, generator adversarial and L1 loss, validation SSIM and L1 (all logged separately in MLflow).

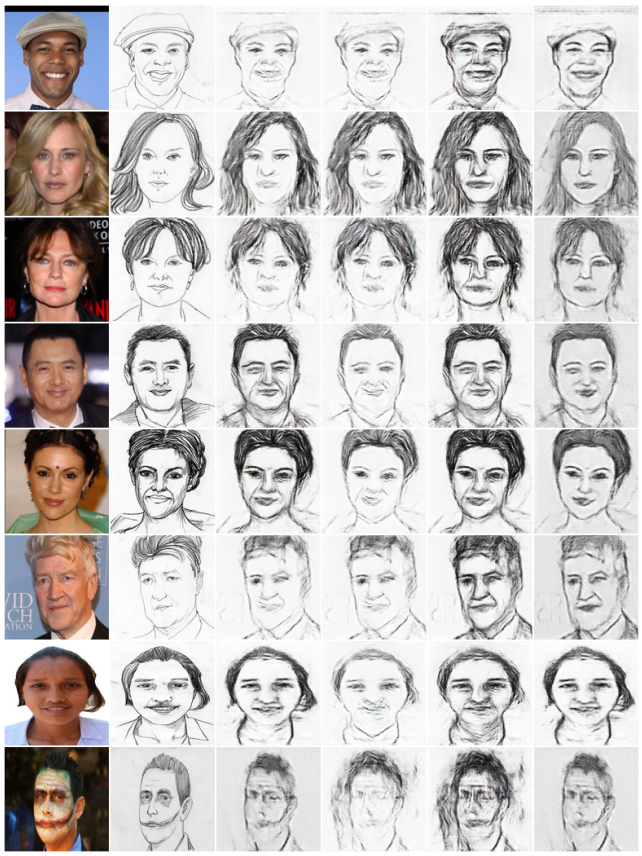


Fig. 14. Fixed validation photographs at epoch 40. Columns: photo, ground truth, generated in its own style, then Style 1, Style 2 and Style 3 for the same photo.

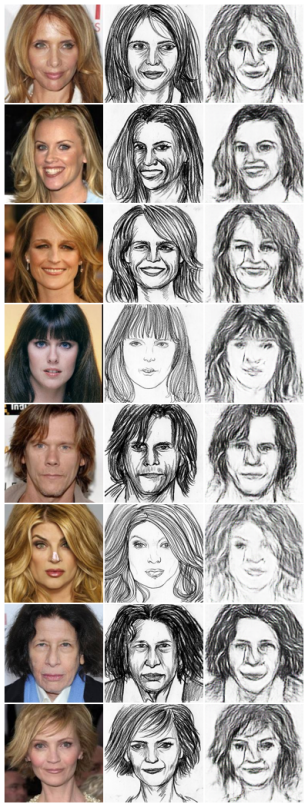


Fig. 15. Lowest-SSIM test cases (photo, ground truth, generated): plausible but misaligned hair strokes.

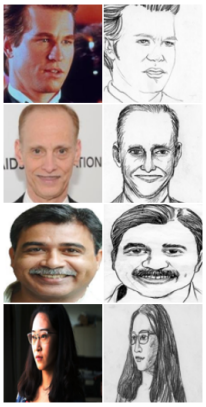


Fig. 16. FS2K training pairs after identical paired augmentation (photo, target sketch).

Stitch screens: see docs/stitch/ in the repository; prompts in docs/stitch_design.md.

Fig. 17. Original Google Stitch design (evidence in the repository).